

9 系统开发与运行

9.1 系统分析与设计

系统分析步骤

- (1) 认识、理解当前的现实环境，获得当前系统的“物理模型”。
- (2) 从当前系统的“物理模型”抽象出当前系统的“逻辑模型”。
- (3) 对当前系统的“逻辑模型”进行分析和优化，建立目标系统的“逻辑模型”。
- (4) 对目标系统的逻辑模型具体化（物理化），建立目标系统的物理模型。

系统开发的目的是将现有系统的物理模型转换为目标系统的物理模型。

系统设计基本原理：抽象；模块化；信息隐蔽；模块独立（高内聚，低耦合，复杂度）。

内聚和耦合

模块的设计要求独立性高，就必须高内聚，低耦合，内聚是指一个模块内部功能之间的相关性，耦合是指多个模块之间的联系，内聚程度从低到高如下表所示：

内聚分类	定义	记忆关键字
偶然内聚	一个模块内的各处理元素之间没有任何联系	无直接关系
逻辑内聚	模块内执行若干个逻辑上相似的功能，通过参数确定该模块完成哪一个功能	逻辑相似、参数决定
时间内聚	把需要同时执行的动作组合在一起形成的模块。	同时执行
过程内聚	一个模块完成多个任务，这些任务必须按指定的过程执行	指定的过程顺序
通信内聚	模块内的所有处理元素都在同一个数据结构上操作，或者各处理使用相同的输入数据或者产生相同的输出数据	相同数据结构、相同输入输出
顺序内聚	一个模块中的各个处理元素都密切相关于同一功能且必须顺序执行，前一个功能元素的输出就是下一个功能元素的输入	顺序执行、输入为输出
功能内聚	最强的内聚，模块内的所有元素共同作用完成一个功能，缺一不可	共同作用、缺一不可

耦合程度从低到高如下所示：

耦合分类	定义	记忆关键字
无直接耦合	两个模块之间没有直接的关系，它们分别从属于不同模块的控制与调用，不传递任何信息	无直接关系
数据耦合	两个模块之间有调用关系，传递的是简单的数据值，相当于高级语言中的值传递	传递数据值调用
标记耦合	两个模块之间传递的是数据结构	传递数据结构
控制耦合	一个模块调用另一个模块时，传递的是控制变量，被调用模块通过该控制变量的值有选择的执行模块内的某一功能	控制变量、选择执行某一功能
外部耦合	模块间通过软件之外的环境联合（如 I/O 将模块耦合到特定的设备、格式、通信协	软件外部环境

	议上) 时。	
公共耦合	通过一个公共数据环境相互作用的那些模块间的耦合	公共数据结构
内容耦合	当一个模块直接使用另一个模块的内部数据，或通过非正常入口转入另一个模块内部时。	模块内部关联

系统结构设计原则：分解-协调原则、自顶向下原则、信息隐蔽和抽象原则、一致性原则、明确性原则、模块间高内聚低耦合、模块的扇入系数和扇出系数合理、模块规模适当。

模块设计

在划分模块时，一个模块的作用范围应该在其控制范围之内。若发现其作用范围不在其控制范围内，应该将判定层次提高，可以将模块合并到父模块，将作用范围模块下移，判定模块上移；但不能将父模块下移，因为此举会让其控制范围变得更小，并且下移父模块也不合理。

9.2 需求分析和需求工程

软件需求概念

按需求内容分类：

业务需求：由客户提出的宏观的一个功能需求。

用户需求：设计员去调查需求中涉及到的每个用户的具体需求。

系统需求：经过整合，形成最终的系统需求，包括功能、性能、设计约束三个方面的需求。

从客户角度分类：

基本需求：需求明确规定的功能。

期望需求：除了基本需求外，客户认为理所应当包含在内的其他功能。

兴奋需求：客户未要求的其他功能需求，会浪费项目开发时间和成本。

软件需求分类

功能需求：软件必须完成的基本动作。

性能需求：说明软件或人与软件交互的静态或动态数值需求，如系统响应速度、处理速度等。

设计约束：受其他标准硬件限制等方面的影响。

属性：可用性、安全性、可维护性、可转移/转移性。

外部接口需求：用户接口、硬件接口、软件接口、通信接口。

9.2.1 需求工程

需求工程六个阶段

需求获取：方法有收集资料、**联合讨论会 JRP**、用户访谈、书面调查、现场观摩、参加业务实践、阅读历史文档、抽样调查。

需求分析与协商：分析不同人提出的所有需求之间的关系并判断。

系统建模：建立系统的抽象模型。

需求规约：也即需求定义，目的是为了编写需求规约（即需求规格说明书），在双方之间达成一个共识。

需求验证：分为两个过程，即需求评审（正式评审和非正式评审）、需求测试（设计概念测试用例）。需求验证通过后，要请用户签字确认，作为验收标准之一，此时，这个需求规格说明书就是需求基线。

需求管理：对需求工程涉及的所有过程进行规划和控制。

需求分析方法

结构化的分析方法 SA，自顶向下，逐步分解，是面向数据的，强调分析对象的数据流，需要建立：功能模型（数据流图）、行为模型（状态转换图）、数据模型（E-R 图）以及数据字典（数据元素、数据结构、数据流、数据存储、加工逻辑、外部实体）。

SA 方法的缺点：人为的将系统分割开来，无法获取各个功能之间的关联情况；若分析的过多，会直接完成设计，即无法区分需求和设计之间的界限。

面向对象的分析方法 OOA，使用 UML 建立模型：构造块（事务、关系、图）、公共机制（规格说明、修饰、公共分类、扩展机制）、规则（对构造块的命名）。

面向数据结构的分析方法。

9.2.2 需求管理

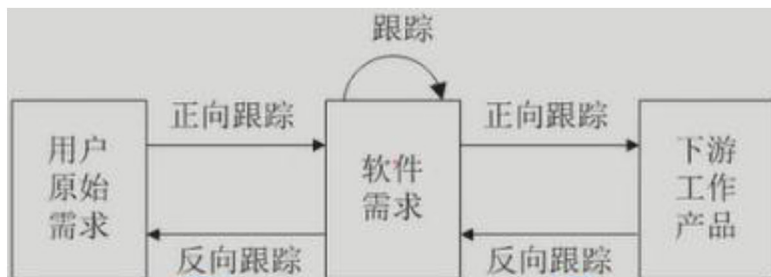
定义需求基线：通过了评审的需求说明书就是需求基线，下次如果需要变更需求，就需要按照流程来一步步进行。

处理需求变更：主要关心需求变更过程中的需求风险管理，带有风险的做法有：无足够用户参与、忽略了用户分类、用户需求的不断增加、模棱两可的需求、不必要的特性、过于精简的 SRS、不准确的估算。

变更产生的原因：外部环境的变化、需求和设计做的不够完整、新技术的出现、公司机构重组造成业务流程的变化。

变更控制委员会 CCB：也称为配置控制委员会，其任务时对建议的配置项变更做出评价、审批，以及监督已经批准变更的实施。

需求跟踪：双向跟踪，两个层次，如下图所示：



正向跟踪表示用户原始需求是否都实现了，反向跟踪表示软件实现的是否都是用户要求的，不多不少。

9.3 结构化分析与设计

属于下午软件设计知识点，详见一本通第三部分：下午软件设计技巧，第1章：数据流图（DFD）。

数据流图（DFD）

用户界面设计的三大原则：置于用户控制之下、减少用户的记忆负担、保持界面的一致性。

9.4 测试基础知识

9.4.1 测试基础

测试原则

应尽早并不断的进行测试；

测试工作应该避免由元开发软件的人或小组承担；

在设计测试方案时，不仅要确定输入数据，而且要根据系统功能确定预期的输出结果；

既包含有效、合理的测试用例，也包含不合理、失效的用例；

检验程序是否做了该做的事，且是否做了不该做的事；

严格按照测试计划进行；

妥善保存测试计划和测试用例；

测试用例可以重复使用或追加测试。

测试类型分为两大类：

动态测试：程序运行时测试，分为

黑盒测试法：功能性测试，不了解软件代码结构，根据功能设计用例，测试软件功能。

白盒测试法：结构性测试，明确代码流程，根据代码逻辑设计用例，进行用例覆盖。

灰盒测试法：即既有黑盒，也有白盒。

静态测试：程序静止时，即对代码进行人工审查，分为

桌前检查：程序员检查自己编写的程序，在程序编译后，单元测试前。

代码审查：由若干个程序员和测试人员组成评审小组，通过召开程序评审会来进行审查。

代码走查：也是采用开会来对代码进行审查，但并非简单的检查代码，而是由测试人员提供测试用例，让程序员扮演计算机的角色，手动运行测试用例，检查代码逻辑。

测试策略

自底向上：从最底层模块开始测试，需要编写驱动程序，而后开始逐一合并模块，最终完成整个系统的测试。优点是较早的验证了底层模块。

自顶向下：先测试整个系统，需要编写桩程序，而后逐步向下直至最后测试最底层模块。优点是较早的验证了系统的主要控制和判断点。

三明治：既有自底向上也有自顶向下的测试方法，二者都包括。兼有二者的优点，缺点是测试工作量大。

9.4.2 测试阶段

单元测试：对单个模块进行测试，由程序员自己测试模块内部的接口、信息、功能，测试依据是软件详细说明书。在单元测试中，**驱动模块（上层）**用来调用被测模块，**自顶向下**的单元测试中不需要另外编写驱动模块。**桩模块（底层）**用来模拟被测模块所调用的子模块。

集成测试：将模块组合起来进行测试，分为一次性组装（简单，节约时间，发现错误少，只适合于小项目）和增量式组装（能够发现更多错误，耗时长，又可分为：自顶向下、自底向上、混合式）。

确认测试：对已完成的软件进行功能上的测试，分为内部确认测试（无用户情况）、**Alpha 测试**（用户在开发环境下进行测试）、**Beta 测试**（用户在实际使用时进行的测试）、验收测试（用户根据SRS对项目进行验收）

系统测试：对软件进行性能测试，主要测试三个方面，即负载测试（在极限情况下，系统各项性能指标）、强度测试（系统资源特别低的情况下）、容量测试（并发测试，系统可以处理的同时在线的最大用户数量）。其他还有可靠性等性能测试，系统测试采用的是黑盒测试方法。

回归测试：软件修改错误或变更后，进行回归测试以验证之前正确的代码是否引入了错误。

9.4.3 测试用例设计

黑盒测试用例：将程序看做一个黑盒子，只知道输入输出，不知道内部代码，由此设计出测试用例，分为下面几类：

等价类划分：把所有的数据按照某种特性进行归类，而后在每类的数据里选取一个即可。等价类测试用例的设计原则：设计一个新的测试用例，使其尽可能多地覆盖尚未被覆盖的有效等价类，重复这一步，直到所有的有效等价类都被覆盖为止；设计一个新的测试用例，使其**仅覆盖一个尚未被覆盖的无效等价类**，重复这一步，直到所有的无效等价类都被覆盖为止。

边界值划分：将每类的边界值作为测试用例，边界值一般为范围的两端值以及在此范围之外的与此范围间隔最小的两个值，如年龄范围为 0-150，边界值为 0,150,-1,151 四个。

错误推测：没有固定的方法，凭经验而言，来推测有可能产生问题的地方，作为测试用例进行测试。

因果图：由一个结果来反推原因的方法，具体结果具体分析，没有固定方法。

白盒测试用例：知道程序的代码逻辑，按照程序的代码语句，来设计覆盖代码分支的测试用例，覆盖级别从低至高分下面六种：

语句覆盖：逻辑代码中的所有语句都要被执行一遍，覆盖层级最低，因为执行了所有的语句，不代表执行了所有的条件判断。

判定覆盖：逻辑代码中的所有判断语句的条件的真假分支都要覆盖一次。

条件覆盖：对于代码中的一个条件，可能是组合的，如 $a > 0 \ \&\& \ b < 0$ ，判断覆盖只针对此组合条件的真假分支做两个测试用例，而条件覆盖是对每个独立的条件都要做真假分支的测试用例，共可有 4 个测试用例，层级更高，注意区别，条件覆盖，针对**每个条件都要真假覆盖**，判定覆盖，**只针对一个条件判断语句**。

判定/条件覆盖：使判定中每个条件的所有可能取值（真/假）至少出现一次，并且每个判定本身的判定结果（真/假）也至少出现一次，即两种覆盖的综合。

条件组合覆盖：每个判定条件中条件的各种可能值的组合都至少出现一次。

路径覆盖：逻辑代码中的所有可行路径都覆盖了，覆盖层级最高。

9.4.4 调试

测试是发现错误，调试是找出错误的代码和原因。

调试需要确定错误的准确位置；确定问题的原因并设法改正；改正后要进行回归测试。

调试的方法有：蛮力法、回溯法（从出错的地方开始，向回找）、**原因排除法**（找出所有可能的原因，逐一进行排除，具体包括演绎法、归纳法、二分法）。

9.5 系统运行与维护

9.5.1 系统转换

系统转换是指新系统开发完毕，投入运行，取代现有系统的过程，需要考虑多方面的问题，以实现与老系统的交接，有以下三种转换计划：

直接转换：现有系统被新系统直接取代了，风险很大，适用于新系统不复杂，或者现有系统已经不能使用的情况。优点是节省成本。

并行转换：新系统和老系统并行工作一段时间，新系统经过试运行后再取代，若新系统在试运行过程中有问题，也不影响现有系统的运行，风险极小，在试运行过程中还可以比较新老系统的性能，适用于大型系统。缺点是耗费人力和时间资源，难以控制两个系统间的数据转换。

分段转换：分期分批逐步转换，是直接和并行转换的集合，将大型系统分为多个子系统，依次试运行每个子系统，成熟一个子系统，就转换一个子系统。同样适用于大型项目，只是更耗时，而且现有系统和新系统间混合使用，需要协调好接口等问题。

数据转换与迁移：将数据从旧数据库迁移到新数据库中。要在新系统中尽可能的保存旧系统中合理的数据结构，才能降低迁移的难度。也有三种方法：系统切换前通过工具迁移、系统切换前采用手工录入、系统切换后通过新系统生成。

转换的过程称为 ETL，有三个步骤：抽取（旧数据库数据）—转换（三种转换方法）—装载（装入新数据库，并校验数据）。

9.5.2 系统维护概述

可维护性的评价指标：

易测试性：指为确认经修改软件所需努力有关的软件属性；

易分析性：指为诊断缺陷或失效原因，或为判定待修改的部分所需努力有关的软件属性；

易改变性：指与进行修改、排错或适应环境变换所需努力有关的软件属性；

稳定性：指与修改造成未预料效果的风险有关的软件属性。

软件维护类型：

正确性维护：发现了 bug 而进行的修改。

适应性维护：由于外部环境发生了改变，被动进行的对软件的修改和升级。

完善性维护：基于用户主动对软件提出更多的需求，修改软件，增加更多的功能，使其比之前的软件功能、性能更高，更加完善。

预防性维护：对未来可能发生的 bug 进行预防性的修改。

9.5.3 系统评价

系统评价分类

立项评价：系统开发前的预评价，分析是否立项开发，做可行性评价。

中期评价：项目开发中期每个阶段的阶段评审。或者项目在开发中途遇到重大变故，评价是否还要继续。

结项评价：系统投入正式运行后，了解系统是否达到预期的目的和要求而对系统进行的综合评价。

系统评价的指标

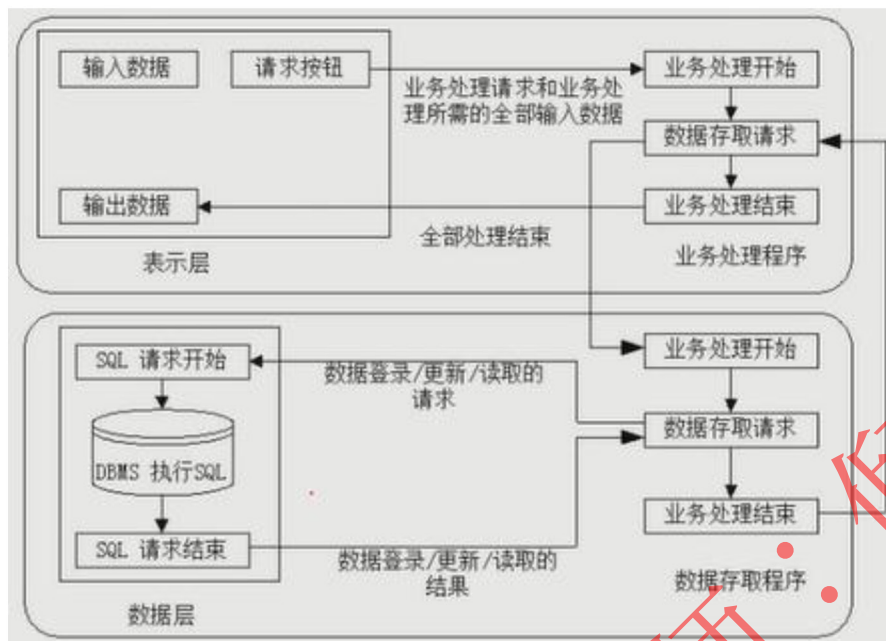
(1) 从信息系统的组成部分出发，信息系统是一个由人机共同组成的系统，所以可以按照运行效果和用户需求（人）、系统质量和技术条件（机）这两条线索构造指标。

(2) 从信息系统的评价对象出发，对于开发方来说，他们所关心的是系统质量和技术水平；对于用户方而言，关心的是用户需求和运行质量；系统外部环境则主要通过社会效益指标来反映。

(3) 从经济学角度出发，分别按系统成本、系统效益和财务指标 3 条线索建立指标。

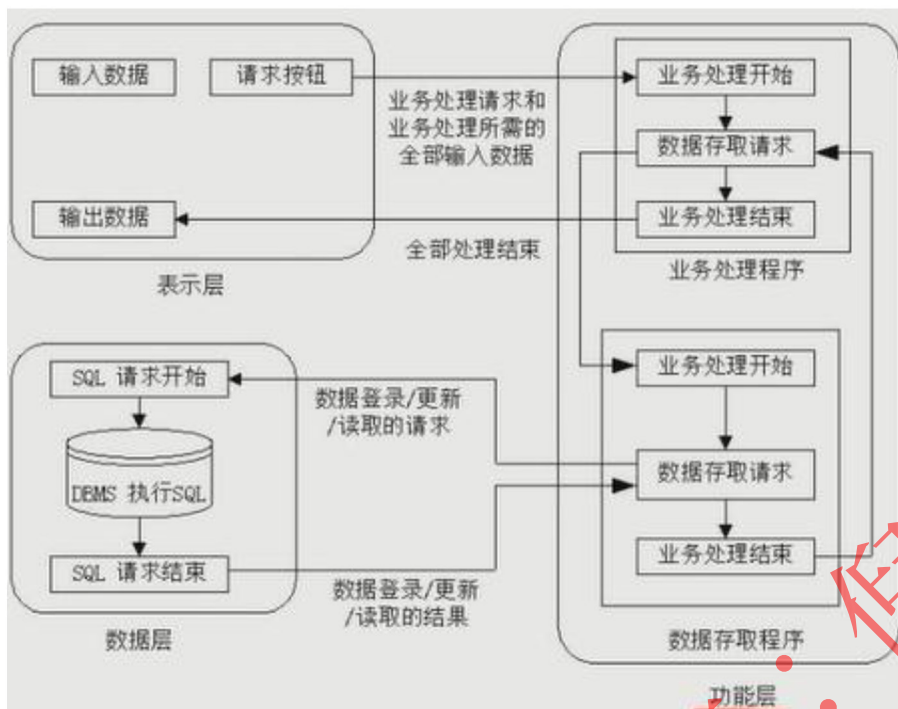
9.6 软件架构介绍

9.6.1 两层 C/S 架构



客户端和服务端都有处理功能，相比较于传统的集中式软件架构，还是有不少优点的，但是现在已经不常用，原因有：开发成本较高、客户端程序设计复杂、信息内容和形式单一、用户界面风格不一、软件移植困难、软件维护和升级困难、新技术不能轻易应用、安全性问题、服务器端压力大难以复用。

9.6.2 三层 C/S 架构



将处理功能独立出来，表示层和数据层都变得简单。表示层在客户机上，功能层在应用服务器上，数据层在数据库服务器上。即将两层 C/S 架构中的数据从服务器中独立出来了。其优点下面四点：

各层在逻辑上保持相对独立，整个系统的逻辑结构更为清晰，能提高系统和软件的可维护性和可扩展性；

允许灵活有效的选用相应的平台和硬件系统，具有良好的可升级性和开放性；

各层可以并行开发，各层也可以选择各自最适合的开发语言；

功能层有效的隔离表示层与数据层，为严格的安全管理奠定了坚实的基础，整个系统的管理层次也更加合理和可控制。

三层 C/S 架构设计的关键在于各层之间的通信效率，要慎重考虑三层间的通信方法、通信频度和数据量，否则即使分配给各层的硬件能力很强，性能也不高。

9.6.3 三层 B/S 架构

是三层 C/S 架构的变种，将客户端变为用户客户端上的浏览器，将应用服务器变为网络上的 WEB 服务器，又称为 0 客户端架构，虽然不用开发客户端，但有很多缺点，主要是数据处理能力差：

B/S 架构缺乏对动态页面的支持能力，没有集成有效的数据库处理功能；

安全性难以控制；

系统开发与运行

1、数据流图(DFD)对系统的功能和功能之间的数据流进行建模，其中顶层数据流图描述了系统的_____。

- A. 处理过程 B. 输入与输出 C. 数据存储 D. 数据实体

答案：B

解析：顶层数据流图只含有一个加工处理表示整个管理信息系统；输入数据流和输出数据流为系统的输入数据和输出数据，表明系统的范围，以及与外部环境的数据交换关系。

2、模块 A 执行几个逻辑上相似的功能，通过参数确定该模块完成哪一个功能，则该模块具有_____内聚。

- A. 顺序 B. 过程 C. 逻辑 D. 功能

答案：C

解析：逻辑内聚是指模块内执行几个逻辑上相似的功能，通过参数确定该模块完成哪一个功能。

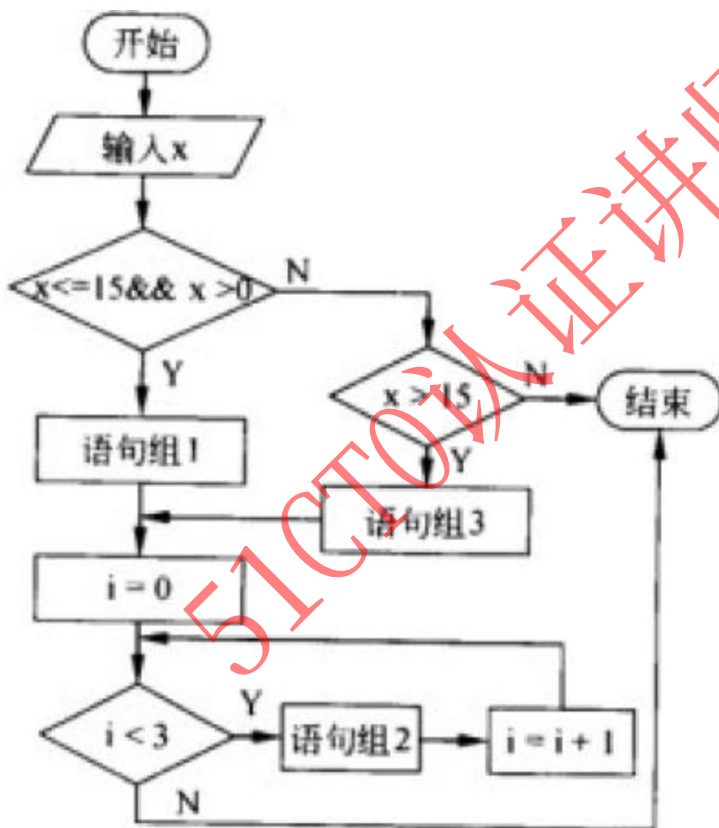
3、某企业由于外部市场环境和管理需求的变化对现有软件系统提出新的需求，则对该软件系统进行的维护属于____维护。

- A. 正确性 B. 完善性 C. 适应性 D. 预防性

答案：C

解析：题目中已经明确说明是因为外部环境变化，因此是适应性维护。适应性维护是指为保持软件产品能在变化后或变化中的环境中可以继续使用而进行的维护活动。

4-5、在白盒测试法中，__4__是最弱的覆盖准则。下图至少需要__5__个测试用例才可以完成路径覆盖，语句组2不对变量i进行操作。



4、 A. 语句 B. 条件 C. 判定 D. 路径

5、 A. 1 B. 2 C. 3 D. 4

答案：A C

解析：白盒测试法的覆盖标准有逻辑覆盖、循环覆盖和基本路径测试。其中逻辑覆盖包括语句覆盖、判定

覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖和路径覆盖等。语句覆盖是指选择足够的测试用例，使被测程序中每条语句至少执行一次。它对程序执行逻辑的覆盖很低，因此一般认为是很弱的逻辑覆盖。

依题意，在图中，要完成路径覆盖至少需要 3 个测试用例才可以。例如，使用测试用例($x=0$)，覆盖{输入 $x \rightarrow (x \leq 15 \& \& x \geq 0) \rightarrow N \rightarrow (x > 15) \rightarrow N \rightarrow \text{结束}$ }路径；使用测试用例($x=8$)，覆盖{输入 $x \rightarrow (x \leq 15 \& \& x \geq 0) \rightarrow Y \rightarrow \text{语句组 1} \rightarrow i=0 \rightarrow i < 3 \rightarrow Y \rightarrow \text{语句组 2} \rightarrow i=i+1 \rightarrow i < 3 \rightarrow Y \rightarrow \dots \rightarrow i < 3 \rightarrow N \rightarrow \text{结束}$ }路径；使用测试用例($x=16$)，覆盖{输入 $x \rightarrow (x \leq 15 \& \& x \geq 0) \rightarrow N \rightarrow (x > 15) \rightarrow Y \rightarrow \text{语句组 3} \rightarrow i=0 \rightarrow i < 3 \rightarrow Y \rightarrow \text{语句组 2} \rightarrow i=i+1 \rightarrow i < 3 \rightarrow Y \rightarrow \dots \rightarrow i < 3 \rightarrow N \rightarrow \text{结束}$ }路径。

6. 以下关于数据流图的叙述中，不正确的是_____。

- A. 每天数据流的起点或终点必须是加工
- B. 必须保持父图与子图平衡
- C. 每个加工必须有输入数据流，但可以没有输出数据流
- D. 应保持数据守恒

答案：C

解析：加工的数据流有输入必然就要有输出，否则是黑洞，其他都是基本规则。

7. 在软件设计阶段，划分模块的原则是：一个模块的_____。

- A. 作用范围应该在其控制范围之内
- B. 控制范围应该在其作用范围之内
- C. 作用范围与控制范围互不包含
- D. 作用范围与控制范围不受限制

答案：A

解析：可以理解，模块的作用范围应该在可控范围内，即在控制范围内，

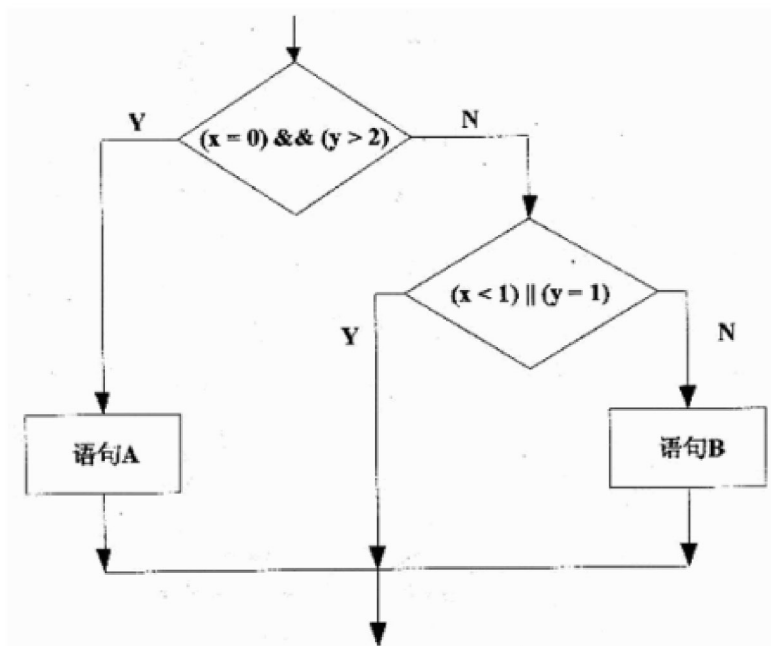
8. 由于信用卡公司升级了其信用卡支付系统，导致超市的原有信息系统也需要做相应的修改工作，该类维护属于_____。

- A. 正确性维护
- B. 适应性维护
- C. 完善性维护
- D. 预防性维护

答案：B

解析：很明显是被动的适应外界环境的变化而做的修改，属于适应性维护。

9-10. 用白盒测试方法对下图所示的程序进行测试，设计了 4 个测试用例：① $(x=0, y=3)$ ，② $(x=1, y=2)$ ，③ $(x=-1, y=2)$ ，④ $(x=3, y=1)$ 。测试用例①②实现了__9__覆盖；若要完成路径覆盖，则可用测试用例__10__。



- (9) A. 语句 B. 条件 C. 判定 D. 路径
(10) A. ①② B. ②③ C. ①②③ D. ①③④

答案：A C

解析：图中共有两条语句，1、2 两个条件正好走遍语句 A 和 B，但是没有覆盖到第二个条件的 Y 分支，也没有对每个条件取值，因此只能是语句覆盖，要完成路径覆盖，只需要在 12 基础上加上一个能走到第二个条件的 Y 分支的条件即可。

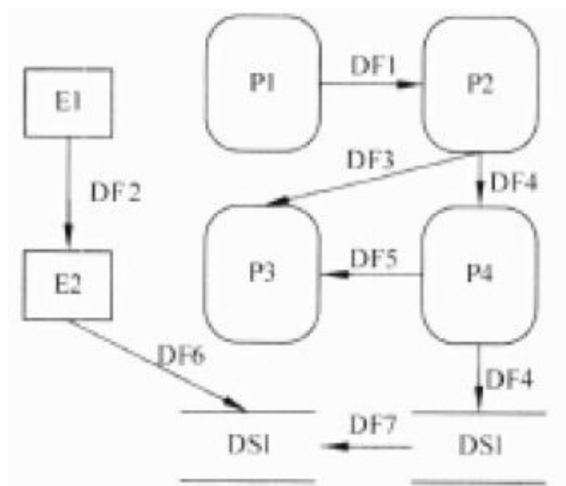
●某项目为了修正一个错误而进行了修改。错误修正后，还需要进行(11)以发现这一修正是否引起原本正确运行的代码出错。

- 11.A.单元测试 B.接受测试 C.安装测试 D.回归测试

答案：D

解析：修改错误后必须进行回归测试，防止修改后引起其他错误。

●在如下所示的数据流图中，共存在 (12) 个错误



12.A.4 B.6 C.8 D.9

答案：B

解析：数据流图有如下几个设计原则：

- (1)数据守恒原则：对任何一个加工来说，其所有输出数据流中的数据必须能从该加工的输入数据流中直接获得，或者说是通过该加工能产生的数据。
- (2)守恒加工原则：对同一个加工来说，输入与输出的名字必须不相同，即使它们的组成成分相同。
- (3)对于每个加工，必须既有输入数据流，又有输出数据流。
- (4)外部实体与外部实体之间不存在数据流
- (5)外部实体与数据存储之间不存在数据流
- (6)数据存储与数据存储之间不存在数据流
- (7)父图与子图的平衡原则：子图的输入输出数据流同父图相应加工的输入输出数据流必须一致，此即父图与子图的平衡。父图与子图之间的平衡原则不存在于单张图。
- (8)数据流与加工有关，且必须经过加工。根据这些原则，DF2 违背了原则 4，DF6 违背了原则 5，DF7 违背了原则 6，P1 和 P3 缺少数据流，违背了原则 3，P4 的输入输出数据流相同，违背了原则 2。因此共有 6 个错误。

●在软件维护中，由于企业的外部市场环境和管理需求的变化而导致的维护工作，属于（13）维护

13.A.正确性 B.适应性 C.完善性 D.预防性

答案：B

解析：明确说明是由于外部市场环境发生变化，因此是适应性维护，适应外界环境变化。

●在设计测试用例时，应遵循（14）原则。

14. A.仅确定测试用例的输入数据，无需考虑输出结果
B.只需检验程序是否执行应有的功能，不需要考虑程序是否做了多余的功能
C.不仅要设计有效合理输入，也要包含不合理、失效的输入
D.测试用例应设计得尽可能复杂

答案：C

解析：常识问题，测试用例的设计要面面俱到，无论有效或无效、输入或输出、简单或复杂等都要考虑。

●单元测试中，检查模块接口时，不需要考虑 (15)

15. A.测试模块的输入参数和形式参数的个数、属性、单位上是否一致
B.全局变量在各模块中的定义和用法是否一致
C.输入是否改变了形式参数
D.输入参数是否使用了尚未赋值或者尚未初始化的变量

答案：D

解析：检查模块的接口，主要是检查模块的输入参数和输出结果，对于输入参数，要检查输入与函数的形参是否一致、是否修改了形参，此外，全局变量也是贯穿多个变量的，但在检查输入参数时，不需要考虑参数是否初始化，即使没有初始化也可以输入，在模块内部初始化再使用。

●以下关于数据流图中基本加工的叙述，不正确的是(16)

- (16) A. 对每一个基本加工，必须有一个加工规格说明
B. 加工规格说明必须描述把输入数据流变换为输出数据流的加工规则
C. 加工规格说明必须描述实现加工的具体流程
D. 决策表可以用来表示加工规格说明

答案：C

解析：数据流图中的加工可以理解为一个活动，输入数据经过转换后输出数据，必须有一个加工规格说明，但无需描述内部具体的流程，只是建立一个概念的数据流走向。

●在划分模块时，一个模块的作用范围应该在其控制范围之内。若发现其作用范围不在其控制范围内，则(17)不是适当的处理方法。

- (17) A. 将判定所在模块合并到父模块中，使判定处于较高层次
B. 将受判定影响的模块下移到控制范围内

- C. 将判定上移到层次较高的位置
- D. 将父模块下移，使该判定处于较高层次

答案：D

解析：判定属于控制范围内，作用范围不在控制范围内，即判定无法控制其作用范围，应该将判定层次提高，可以将模块合并到父模块，将作用范围模块下移，判定模块上移，但将父模块下移，实际是将判定范围下移，其控制范围会变得更小，并且下移父模块也不合理。

● 系统可维护性的评价指标不包括(18)。

- (18)A. 可理解性 B. 可测试性 C. 可移植性 D. 可修改性

答案：C

解析：可维护，可以从意思理解，若需要维护，必然要理解软件、而后修改软件，修改后要能测试正确性，与移植与否无关，软件质量属性繁多，要真正理解才不会混淆，在文老师视频课程会详细讲解。

● 逆向工程从源代码或目标代码中提取设计信息，通常在原软件生命周期的(19)阶段进行。

- (19)A. 需求分析 B. 软件设计 C. 软件实现 D. 软件维护

答案：D

解析：由题意，逆向工程从源程序提取信息，前提是软件已经开发完成，因此是在维护阶段。

● 一个程序根据输入的年份和月份计算该年中该月的天数，输入参数包括年份(正整数)、月份(用 1~12 表示)。若用等价类划分测试方法测试，则(20)不是一个进行合适的测试用例(分号后表示测试的输出)。

- (20)A. (2013, 1; 31) B. (0, 1; '错误')
C. (0, 13; '错误') D. (2000, -1; '错误')

答案：C

解析：等价类划分，就是一个测试用例可以代表一类条件，A 是正确的情况，B 是年份错误月份正确情况，D 是年份正确月份错误情况，而 C 是年份和月份都错误的情况，覆盖了两个无效等价类，不适合，设计一个测试用例，应该只覆盖一个无效等价类。

● (21) 不是单元测试主要检查的内容。

- (21)A. 模块接口 B. 局部数据结构 C. 全局数据结构 D. 重要的执行路径

答案：C

解析：单元测试是模块内部测试，主要测试模块内部数据流、控制流、局部数据结构、接口等单个模块相关，全局相关的在集成测试和系统测试进行。

●以下关于结构化开发方法的叙述中，不正确的是 (22) 。

- (22) A. 将数据流映射为软件系统的模块结构
B. 一般情况下，数据流类型包括变换流型和事务流型
C. 不同类型的数据流有不同的映射方法
D. 一个软件系统只有一种数据流类型

答案：D

解析：结构化开发方法的特点是自顶向下，面向数据流的，但是不可能限制数据流类型，可以有多种数据流类型，也可以用自相矛盾法排除，如 BC 都是说明多类型数据流，那么肯定可以有多类型数据流。

●模块 A 提供某个班级某门课程的成绩给模块 B，模块 B 计算平均成绩、最高分和最低分，将计算结果返回给模块 A，则模块 B 在软件结构图中属于 (23) 模块。

- (23) A. 传入 B. 传出 C. 变换 D. 协调

答案：C

解析：传入模块：从下属模块取得数据，经处理再将其传送给上级模块。 传出模块：从上级模块取得数据，经处理再将其传送给下属模块。 变换模块：从上级模块取得数据，进行特定的处理，转换成其他形式，再传送给上级模块。协调模块：对所有下属模块进行协调和管理的模块。由题意可知，模块 B 从上级模块 A 取得数据，经过处理又传送给上级模块 A，属于变换模块。

●某搜索引擎在使用过程中，若要增加接受语音输入的功能，使得用户可以通过语音输入来进行搜索，此时应对系统进行 (24) 维护。

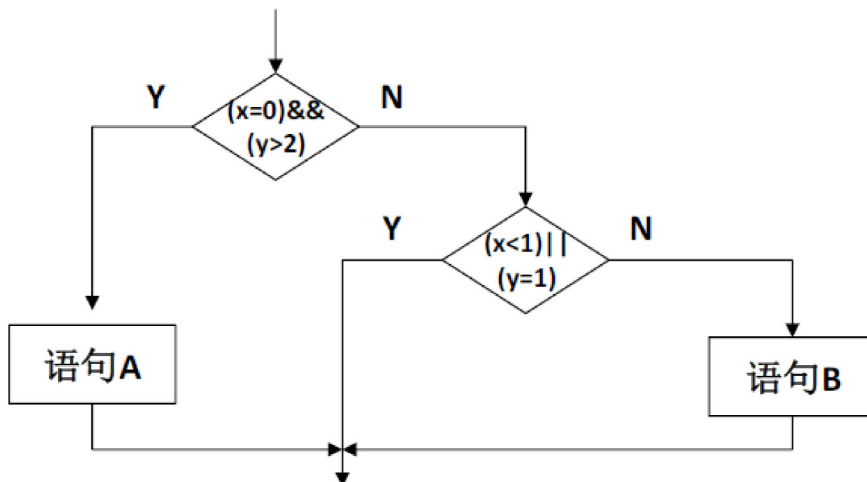
- (24) A. 正确性 B. 适应性 C. 完善性 D. 预防性

答案：B

解析：由题意，既不是有错误也不是预防未来，因此 AD 排除，BC 容易混淆，但要记住，适应性是适应环境的变化，对软件进行修改和升级，而完善性是用户主动提出需求，增加软件功能和性能，理解题意，可知更偏向于使用时外部环境发生改变，需要增加语音输入功能。

●采用白盒测试方法对下图进行测试，设计了 4 个测试用例：①($x=0, y=3$)，②($x=1, y=2$)，③($x=-1$ ，

y=2), ④(x=3,y=1)。至少需要测试用例①②才能完成 (25) 覆盖, 至少需要测试用例①②③或①②④才能完成 (26) 覆盖。



- (25) A. 语句 B. 条件 C. 判定 / 条件 D. 路径
(26) A. 语句 B. 条件 C. 判定 / 条件 D. 路径

答案: A D

解析: 可根据问题将测试用例代入执行, ①②会执行语句 A 和语句 B, 但只覆盖第一个条件, 因此是语句覆盖; ①②覆盖了第一个条件的真假, 执行了第一个条件的 Y 和 N 以及第二个条件的 N, 而③或者④又执行到了第二个条件的 Y, 因此流程中所有分支都走到了, 是路径覆盖。

27、以下关于结构化开发方法的叙述中, 不正确的是 ()。

- A. 总的指导思想是自顶向下, 逐层分解
B. 基本原则是功能的分解与抽象
C. 与面向对象开发方法相比, 更适合于大规模、特别复杂的项目
D. 特别适合于数据处理领域的项目

答案: C

解析: 可以使用排除法, 结构化开发方法是面向数据流, 自顶向下、逐层分解的, 因此 ABD 都对, 但目前已经逐渐被面向对象开发方法替代, 因为面向对象开发方法更适合大型复杂项目。

28、模块 A、B 和 C 都包含相同的 5 个语句, 这些语句之间没有联系。为了避免重复把这 5 个语句抽取出来组成一个模块 D, 则模块 D 的内聚类型为 () 内聚。

- A. 功能 B. 通信 C. 逻辑 D. 巧合

答案: D

解析：明确说明这这些语句之间没有任何联系，只是简单的冗余，因此是偶然（巧合）内聚。

29、在软件开发过程中，系统测试阶段的测试目标来自于（）阶段。

- A. 需求分析 B. 概要设计 C. 详细设计 D. 软件实现

答案：A

解析：测试的目标是需求分析开始。从需求开始时就已经确定了产品的功能，在哪些的用户场景、用户使用的功能。

30、以下关于软件维护和可维护性的叙述中，不正确的是（）。

- A. 软件维护要解决软件产品交付用户之后运行中发生的各种问题
B. 软件的维护期通常比开发期长得多，其投入也大得多
C. 进行质量保证审查可以提高软件产品的可维护性
D. 提高可维护性是在软件维护阶段考虑的问题

答案：D

解析：在软件开发的各个阶段都需考虑软件的可维护性，这是一个套路，一般涉及到软件质量、性能特性的，都需要在软件各个阶段考虑。

面向对象技术

1、面向对象技术中，组合关系表示_____。

- A. 包与其中模型元素的关系 B. 用例之间的一种关系
C. 类与其对象的关系 D. 整体与其部分之间的一种关系

答案：D

解析：组合关系表示整体与其部分之间的一种关系。包用于将关系紧密的模型元素组织在一起，提供一个命名空间，以提供访问控制。用例之间有包含(include)、扩展(extend)和泛化(generalization)3种相关性的关系。类是在对象之上的抽象，对象是类的具体化，对定义好的类的属性的不同赋值就可以得到该类的对象实例。

2、以下关于封装在软件复用中所充当的角色的叙述中，正确的是_____。